Curs 5a

Programare Orientată pe Obiecte în limbajul Java

Programare Orientată pe Obiecte

Cuprins

- Upcasting/Downcasting exemplu
- Exceptii

Quizz

```
class Patrat extends Poligon { … }
```

Varianta 1:

```
Patrat metoda2( ) {
    Poligon p = new Poligon();
    Patrat t = new Patrat();
    if (...)
        return p; // Eroare!!
    else
        return t; // Corect
}
```

Varianta 2:

```
Patrat metoda2( ) {
    Poligon p = new Poligon();
    Patrat t = new Patrat();
    if (...)
        return (Patrat)p; // corect?  Se execută?
        /*daca nu e corect, cum putem corecta fara sa
          schimbam tipul lui p */
    else
        return t; // Corect
    }
```

Excepţii

- Ce sunt excepţiile
- ”Prinderea” şi tratarea excepţiilor
- ”Aruncarea” excepţiilor
- Avantajele tratării excepţiilor
- Ierarhia claselor ce descriu excepţii
- Excepţii la execuţie
- Crearea propriilor excepţii

Ce sunt excepţiile ?

Excepţie = ”eveniment excepţional”

```
public class Exemplu {
   public static void main(String args[]) {
     int v[] = new int [10];
     v [10] = 0; //Exceptie !
     System.out.println("Aici nu se mai ajunge..");
   }
}
```

"Exception in thread "main"
java.lang.ArrayIndexOutOfBoundsException :10
at excepţii.main (excepţii.java:4)“

- ”throw an exception”
- ”exception handler”
- ”catch the exception”

Tratarea erorilor nu mai este o opţiune ci o constrângere!

”Prinderea” şi tratarea excepţiilor

try - catch - finally

```
try {
    … // Instrucţiuni care pot genera excepţii
}
catch (TipExceptie1 variabila) {
    … // Tratarea excepţiilor de tipul 1
}
catch (TipExceptie2 variabila) {
    … // Tratarea excepţiilor de tipul 2
}
. . .
finally {
    … // Cod care se execută indiferent
    … // dacă apar sau nu excepţii
}
```

Citirea unui fişier (1)

```
public static void citesteFisier(String fis) {
      FileReader f = null;
      // Deschidem fisierul
      f = new FileReader(fis);
      // Citim si afisam fisierul caracter cu
      //  caracter
      int c;
      while ( (c=f.read()) != -1)
            System.out.print((char)c);
      // Inchidem fisierul
      f.close();
}
```

Pot provoca excepţii:

- Constructorul lui FileReader
- read
- close

Citirea unui fişier (2)

```
public static void citesteFisier(String fis) {
      FileReader f = null;
      try {
                  // Deschidem fisierul
            f = new FileReader(fis);
                  // Citim si afisam fisierul caracter cu caracter
            int c;
            while ( (c=f.read()) != -1)
              System.out.print((char)c);
      }
      catch (FileNotFoundException e) {
            //Tratam un tip de exceptie
         System.err.println("Fisierul nu a fost gasit");
      }
      catch (IOException e) {
            //Tratam alt tip de exceptie
         System.out.println("Eroare la citire");
         e.printStackTrace();
      }
```

Citirea unui fişier (3)

```
        finally {
           if (f != null) {
            // Inchidem fisierul
              try {
                f.close();
              }
              catch (IOException e) {
                  System.err.println("Fisierul nu poate fi
           inchis!");
                  e.printStackTrace(); }
            } // if
        } //finally
}
```

”Aruncarea” excepţiilor (1)

- A doua metodă de lucru cu excepţiile
- Se utilizează clauza throws în antetul metodelor care pot genera excepţii:

```
[modific] TipReturnat metoda([argumente])
    throws TipExceptie1, TipExceptie2, ...
{
...
}
```

Atentie !!!

- **O metoda care nu tratează o anumita exceptie trebuie obligatoriu să o ”arunce”.**

”Aruncarea” excepţiilor (2)

```
public class CitireFisier {
   public static void citesteFisier(String fis) throws
    FileNotFoundException, IOException
    {
       FileReader f = null;
       f = new FileReader(fis);
       int c;
       while ( (c=f.read()) != -1)
           System.out.print((char)c);
       f.close();
    }
```

”Aruncarea” excepţiilor (3)

```
  public static void main(String args[]) {
     if (args.length > 0) {
          try {
               citesteFisier(args[0]);
          }
          catch (FileNotFoundException e){
               System.err.println("Fisierul n-a fost gasit");
           }
          catch (IOException e) {
               System.out.println("Eroare citire/inchidere");
          }
     }
     else
          System.out.println("Lipseste numele fisierului");
  } // main
} // clasa
```

try - finally

```
public static void citesteFisier (String fis) throws
    FileNotFoundException, IOException //Amandoua?
{
     FileReader f = null;
     try {
          f = new FileReader (fis);
          int c;
          while ( (c=f.read()) != -1)
               System.out.print((char)c);
     }
     finally {
          if (f!=null)
               f.close();
     }
}
public static void main (String args[]) throws
    FileNotFoundException, IOException {
                    citesteFisier(args[0]);
}
```

Instrucţiunea throw

- Aruncarea explicită de excepţii:

Exemplu:

```
throw new IOException("Exceptie I/O");
```

Sau:

```
if (index >= vector.length)
    throw new
       ArrayIndexOutOfBoundsException();
```

Sau:

```
catch(Exception e) {
    System.out.println ("A aparut o exceptie);
    throw e;
}
```

Avantajele tratării excepţiilor

1. Separarea codului
2. Propagarea erorilor
3. Gruparea erorilor după tip.

Separarea codului (1)

```
citesteFisier {
    deschide fişierul;
    determină dimensiunea fişierului;
    alocă memorie;
    citeşte fişierul în memorie;
    închide fişierul;
}
```

Separarea codului (2)

Cod ”tradiţional” (”spaghetti”):

```
int citesteFisier() {
    int codEroare = 0;
    deschide fisierul;
    if (fisierul s-a deschis) {
        determina dimensiunea fisierului;
        if (s-a determinat dimensiunea) {
            aloca memorie;
            if (s-a alocat memorie) {
                citeste fisierul in memorie;
                if (nu se poate citi din fisier) {
                    codEroare = -1;
                }
        } else { ...
}}}}}}}}}}}}}}}}}}}}}}}}}}}}}}}
return codEroare; }
```

Separarea codului (3)

```
int citesteFisier() {
        try {
                deschide fişierul;
                determină dimensiunea fişierului;
                alocă memorie;
                citeşte fişierul în memorie;
                închide fişierul;
        }
        catch (fişierul nu s-a deschis)
                {tratează eroarea;}
        catch (nu s-a determinat dimensiunea)
                {tratează eroarea;}
        catch (nu s-a alocat memorie)
                {tratează eroarea}
        catch (nu se poate citi din fişier)
                {tratează eroarea;}
        catch (nu se poate închide fişierul)
                {tratează eroarea;}
}
```

Propagarea erorilor

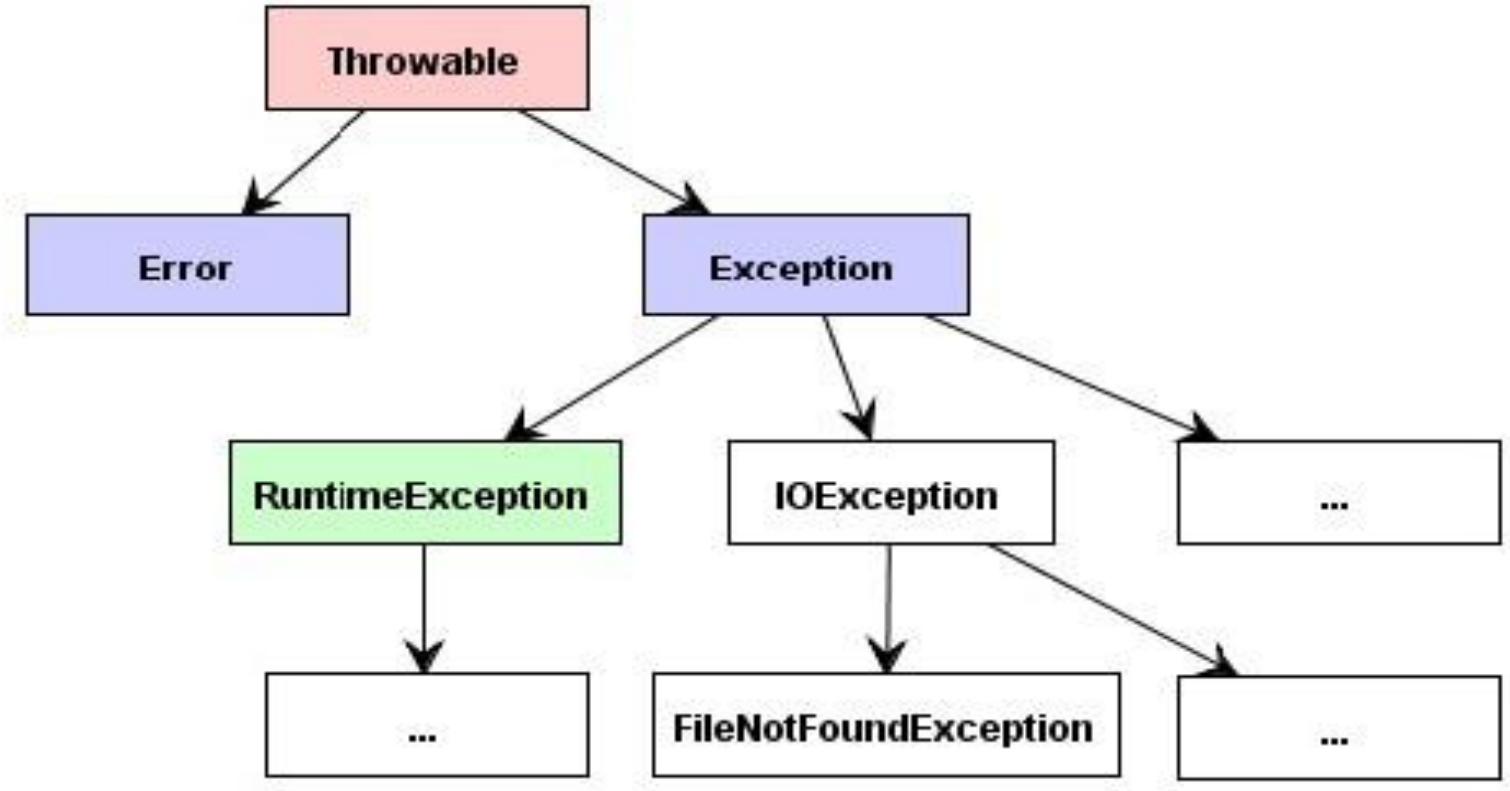
```
int metoda1() {
	try {
		metoda2();
	}
	catch (TipExceptie e) {
		//proceseazaEroare;
	}
	...
}
int metoda2() throws TipExceptie {
	metoda3();
	...
}
int metoda3() throws TipExceptie {
	citesteFisier();
	...
}
```

Gruparea erorilor după tipul lor

- Fiecare tip de excepţie este descris de o clasă.
- Clasele sunt organizate ierarhic.

```
try {
      FileReader f = new FileReader("input.dat");
      // Exceptie posibilă: FileNotFoundException
}
catch (FileNotFoundException e) {
      // Exceptie specifică provocată de absenţa
      // fişierului ’input.dat’
} // sau
catch (IOException e) {
      // Exceptie generică provocată de o operatie IO
} // sau
catch (Exception e) {
      // Cea mai generică excepţie soft
} //sau
catch (Throwable e) {
      // Superclasa excepţiilor
}
```

Ierarhia claselor ce descriu excepţii



Metode:

- getMessage
- printStackTrace
- toString

Excepţii la execuţie

RuntimeException

- ArithmeticException
- NullPointerException
- ArrayIndexOutOfBoundsException

…

```
int v[] = new int[10];
try {
      v[10] = 0; ..
}
catch (ArrayIndexOutOfBoundsException e) {
      System.out.println("Atentie la indecsi!");
      e.printStackTrace();
} // Corect, programul continuă

v[11] = 0;
/* Nu apare eroare la compilare dar apare exceptie la
    executie si programul va fi terminat.*/
System.out.println("Aici nu se mai ajunge...");
```

ArithmeticException

- Împărţirea la 0 va genera o excepţie doar dacă tipul numerelor împărţite este aritmetic întreg.
- În cazul tipurilor reale (float şi double) nu va fi generată nici o excepţie, ci va fi furnizat ca rezultat o constantă care poate fi, funcţie de operaţie, Infinity, -Infinity, sau NaN.

```
int a=1, int b=0;
System.out.println(a/b); // Exceptie la executie!
double x=1, y=-1, z=0;
System.out.println(x/z); // Infinity
System.out.println(y/z); // -Infinity
System.out.println(z/z); // NaN
```

Crearea propriilor excepţii (1)

```
public class ExceptieProprie extends
    Exception {
     public ExceptieProprie(String mesaj) {
          super(mesaj);
          /* Apeleaza constructorul superclasei
            Exception */
     }
}
```

Exemplu:

```
class ExceptieStiva extends Exception {
     public ExceptieStiva(String mesaj) {
          super(mesaj);
     }
}
```

Crearea propriilor excepţii (2)

```
class Stiva {
    int elemente[] = new int[100];
    int n=0; //numarul de elemente din stiva
    public void adauga(int x) throws ExceptieStiva {
        if (n==100)
            throw new ExceptieStiva("Stiva este plina!");
        elemente[n++] = x;
    }
    public int scoate() throws ExceptieStiva {
        if (n==0)
            throw new ExceptieStiva("Stiva este
          goala!");
        return elemente[--n];
    }
}
```

Intrări formatate: java.util.Scanner

```
Scanner s=new Scanner(System.in);

String nume = s.next();
int varsta = s.nextInt();
double salariu = s.nextDouble();

s.close();
```